

PLATAFORMA DE OPERACIÓN DE UNA CONSULTORA TECNOLÓGICA

Vende servicios.

La Dell no es un servidor. Es el Centro de Operaciones.

Cada proyecto es un laboratorio.

Debe tener:

- documentación
- video
- código
- docker
- base de datos
- presentación
- dashboard
- métricas
- demostración

Cuando un cliente diga "Muéstreme Quesinor" Levanto el laboratorio.

El laboratorio no demuestra conocimientos. Demuestra cómo trabajo.

La demostración

No quiero enseñar código.

Quiero enseñar operación

tres escenarios.

Escenario 1 Trabajar desde el laboratorio

Estás en el laboratorio.

Todo local. (Zoom.Dell.Lenovo.Surface)

Escenario 2 Trabajar desde cualquier lugar.

Estoy fuera.(MacBook.iPad.Tailscale.Grafana.SSH.Todo remoto.)

Escenario 3 yo no puedo operar

No estoy.(Otra persona debe operar.Por eso existe el Manual de Gobierno. NO depende de una persona, Es continuidad operative , pensamiento empresarial)

Evidencia antes que suposición expuesto en el manual de arquitectura y gobierno

Esa frase debería gobernar absolutamente todo.

significa que cuando trabajemos: Nunca asumirás nada. Nunca cambiaras algo porque "parece mejor". Siempre iras por evidencia.

Centro de Operaciones : operations.lab (allí existan botones.
Portfolio,Wiki,Grafana,Portainer,Gitea,OpenWebUI,Node
Red,N8N,Prometheus,Kuma,MinIO,Logs,Backups,Cientes,Proyectos,Laboratorios,Alertas,
Calendario,SSH,Estado del servidor,Todo.Un único punto de entrada) con el tiempo escala
a mas de 40 sera imposible

PORTFOLIO. Primero (Versión ,
Estable,Documentada,Segura,HTTPS,Analytics,SEO,Automatizaciones,Git,Deploy,Backups
)

Quiero usar el portfolio como **caso de prueba**. Porque cuando llegue el proyecto real del cliente,,,, ya sabre exactamente cómo desplegarlo.

Por el momento esta en desarrollo se realizaría pruebas con dominio gratuito temporal
diversificar la presencia:

- **LinkedIn** para networking profesional y visibilidad.
- **GitHub** como evidencia técnica del código y documentación.
- **Dev.to** para artículos técnicos sobre tus laboratorios.
- **Medium** para casos de estudio más largos.
- **Hashnode** para documentación orientada a desarrolladores.
- **YouTube** con demostraciones cortas de proyectos y despliegues.
- **Reddit** (comunidades técnicas específicas) para compartir aprendizajes cuando aporte valor.
- **Portfolio** como el punto central al que todo lo anterior dirige tráfico.

La idea es que ninguna plataforma sea "mi negocio". mi negocio debe vivir en mi propio dominio.

El reto será construir una plataforma empresarial.

¿Aporta valor al negocio?

- ¿Respetar el Manual de Arquitectura y Gobierno?
- ¿Es reproducible y documentable?
- ¿Mejora la experiencia del cliente o del operador?

Si una propuesta no supera esas preguntas, no merece entrar en producción.

Esto debe ser

Será convertir cada proyecto en un producto operativo.

Por ejemplo. Quesinor.(Hoy tengo, PDF, Video demo, Wiki, Estudio de caso, Código, Base de datos, etc)

Nuestro trabajo será convertir eso en un laboratorio reproducible.

Algo como: portfolio , Qesinor, docker compose, Wiki, Repositorio, Video completo no demo, Dashboard, Logs, Backup, Monitoreo, Manual, Caso de estudio, Presentación, Evidencias)

Cuando un cliente diga: "ejecútelo" , No buscare archivos. Simplemente levantarás este laboratorio (Qesinor)

De lo anterior resulta por qué ese contenedor existe dentro de Mi plataforma.

Ejemplo. Grafana. No será "cómo instalar Grafana"

Será

¿Por qué Grafana está aquí?

¿Qué información necesita un cliente?

¿Qué información necesita un reclutador?

¿Qué información necesitas tú?

¿Qué paneles deben existir?

¿Cuáles sobran?

¿Qué alertas justifican enviar un mensaje al teléfono?

¿Qué métricas realmente aportan valor?

Lo mismo con: Prometheus. Loki, Portainer, n8n, etc , etc

Las configuraciones **en función de la plataforma**, usando el portfolio como primer caso de uso.

¿Por qué?

Porque mañana el portfolio será un proyecto más.

Después vendrá Quesinor.

Después Compras Públicas.

Después un proyecto real de un cliente real

Si toda la plataforma queda diseñada alrededor del portfolio (eso no quiero), el día que aparezca un proyecto distinto tendría que romper el diseño.

Si la plataforma se diseña alrededor del concepto de "laboratorio", el portfolio solo será el Laboratorio 0

orden correcto empieza por GitHub

No por subir el portfolio. Sino por **hacer que sea publicable de forma segura**.

Preparación para GitHub

Auditoría completa.

Objetivo

Que el repositorio pueda hacerse público sin exponer:

- credenciales
- API Keys
- endpoints privados
- estructura interna
- IP internas
- rutas privadas

- secretos
- configuración del asistente

Aquí entran: (.env, env.example, gitignore, README, LICENSE, estructura)

quedará documentado como una **regla de la plataforma**, no solo como una tarea técnica

Revisión completa del portfolio. (El archivo tiene muchísimo JavaScript embebido, Tiene CSS enorme embebido, Tiene lógica 3D, Tiene chatbot., Tiene traducciones. Tiene comentarios. Tiene citas. Tiene reconocimiento por voz. Tiene muchas responsabilidades en un único archivo. Etc, etc) Significa que antes de migrarlo al Dell debo tener el definitivo

GitHub: Cuando el proyecto sea segur,

Hosting temporal, Migración Dell, Laboratorios, Automatizaciones, Producción

tomar cuenta que : En el documento de reestructuración aparece claramente que la metodología empieza por: Baseline → Inventario → Documentación → Consistencia → RFC → Implementación

el portfolio no ha pasado por una revisión completa de baseline. Rectificar

hacer una auditoría complete no solamente de seguridad también: arquitectura , mantenibilidad , despliegue , SEO , rendimiento , automatización , integración con la plataforma , integración con Docker , integración con Homer , integración con n8n , integración con Wiki , integración con Grafana de aquí saldrá una lista de RFC

En lugar de saltar directamente a GitHub, elaborar primero un documento llamado:

RFC-PORT-001 – Auditoría del Portfolio

El objetivo **no es corregir código todavía**.

identificaríamos **todo** lo que impide que ese portfolio sea un activo de producción: ¿Está este portfolio listo para convertirse en el núcleo de la plataforma y publicarse de forma segura?

- qué está bien,
- qué es mejorable,
- qué representa un riesgo,
- qué debe ocultarse antes de publicar en GitHub,
- qué dependencias deben externalizarse,
- qué puede afectar la seguridad o el mantenimiento.

1. Arquitectura

- Estructura de carpetas.
- Separación HTML / CSS / JS.
- Organización de componentes.
- Dependencias.
- Flujo de navegación.
- Escalabilidad.

2. Seguridad

Este será probablemente el apartado más importante.

Buscar, entre otras cosas:

- credenciales
- tokens
- API Keys
- secretos
- rutas internas
- IP privadas
- dominios internos
- endpoints
- conexiones a BD
- configuración del chatbot
- configuración de Calendly
- WhatsApp
- Google Analytics
- variables sensibles

El resultado será algo como:

Riesgo	Estado	Acción
Variables sensibles	✗	Pasar a .env
Configuración chatbot	✗	Externalizar
Datos públicos	✓	Mantener

Preparación para GitHub

Antes de subir nada verificar:

- .gitignore
- .env
- .env.example
- estructura del repositorio
- ramas
- README
- LICENSE
- documentación mínima
- historial de Git (para evitar subir secretos por accidente)

Rendimiento

Revisar:

- tamaño del HTML
- CSS
- JavaScript
- imágenes
- videos
- lazy loading
- compresión
- caché

SEO

Aunque no sea la prioridad, es importante para un portfolio público.

Revisar:

- meta tags
- Open Graph
- favicon
- robots
- sitemap
- schema.org
- títulos
- accesibilidad

Integración con la plataforma

Este punto no suele aparecer en auditorías tradicionales, pero en mi caso es fundamental.

revisar si el portfolio está preparado para integrarse con:

- Wiki.js
- Grafana

- Prometheus
- Portainer
- Homer
- n8n
- Node-RED
- Gitea/GitHub
- Base de datos
- Asistente virtual

Experiencia del cliente:

- ☐ ¿Qué entiende un CEO en 30 segundos?
- ☐ ¿Qué entiende un reclutador?
- ☐ ¿Qué entiende un CTO?
- ☐ ¿Qué entiende un cliente que quiere contratar una consultoría?

Documentación

Todo hallazgo terminará documentado.

Diremos:

- descripción
- evidencia
- impacto
- prioridad
- propuesta
- decisión adoptada

De esa forma quedará alineado con el Manual

Cómo entregar los resultados

Auditoría: formato similar a este:

1. Resumen ejecutivo.
2. Hallazgos críticos (resolver antes de GitHub).
3. Hallazgos importantes.

4. Hallazgos recomendados.
5. Lo que está bien implementado.
6. Roadmap de corrección.
7. Criterios para considerar el portfolio "listo para producción".

Hallazgo Crítico 001

La configuración está dentro del proyecto.

Acabo de encontrar esto:

```
define('DB_HOST', 'localhost');  
define('DB_NAME', 'portfolio_db');  
define('DB_USER', 'root');  
define('DB_PASS', "");
```

Está en

api/config.php

Eso significa que hoy mismo el proyecto conoce:

- servidor
- base de datos
- usuario
- contraseña

Hoy la contraseña está vacía.

Mañana estará llena.

Y el día que hagas

git push

quedará registrada para siempre en el historial.

Aunque luego la borres.

Eso es exactamente el tipo de problema que querías evitar.

Lo que significa

No hay separación entre

Código

y

Configuración.

Y eso rompe uno de los principios fundamentales de cualquier despliegue serio

Bajo esta hallazgos se define el estándar de la plataforma.

Platform Rules

- Nunca habrá credenciales dentro del código.
- Toda configuración vive en .env
- Nunca se subirá .env
- Siempre existirá .env.example
- Toda API Key será variable de entorno.
- Toda IP privada será variable de entorno.
- Todo endpoint privado será variable de entorno.

esto pasa a formar parte del Manual. Es una regla permanente

el portfolio es mucho más grande de lo que esperaba tiene:(admin/, api/, assets/, pdf/, videos/, crm/, chatbot, appointments, comments , likes, dashboard, etc, etc.) esto ya es considerado como una aplicación

tiene una separación lógica: admin , api, assets, videos, pdf (no hay que reinventar el proyecto)

como vez tiene un: admin/ por ende existe BackOffice.

Entonces inmediatamente aparecen preguntas.

¿Está protegido?

¿Tiene sesiones?

¿CSRF?

¿Rate Limit?

¿Brute Force?

¿2FA?

¿Logs?

Esto entra automáticamente en la auditoría.

También tiene api/ :

revisar

- quién consume esas APIs
- autenticación
- validación
- sanitización
- SQL Injection
- XSS
- CORS
- CSRF
- headers

Todavía no emitir juicio hasta revisarlas por ello merece una **auditoría profesional**, similar a la que se haría antes de poner en producción una aplicación.

No se aceptarán cambios arquitectónicos que solo respondan a preferencias personales o modas tecnológicas. Si algo se cambia, será porque aporta una mejora demostrable.

Ya que veo el portfolio como un sitio web pero también lo veo como el **primer producto** de mi plataforma.

Eso significa revisar con el mismo nivel de exigencia que revisaría una aplicación que va a ser usada por una empresa.

Y eso implica revisar también cosas que normalmente nadie revisa:

- consistencia arquitectónica,
- deuda técnica,
- mantenibilidad,

- desacoplamiento,
- capacidad de crecer cuando tengas 10, 20 o 50 laboratorios,
- facilidad de migración entre tu Lenovo i9 y la Dell,
- integración futura con Homer, Wiki.js, Grafana, n8n y Node-RED.

Quiero que cada hallazgo termine en un **RFC** con:

- Evidencia.
- Riesgo.
- Impacto.
- Prioridad.
- Alternativas.
- Decisión arquitectónica.
- Plan de implementación.

Es más lento, pero cuando terminemos no solo tendría un portfolio mejor: tendría un conjunto de decisiones documentadas que servirán como base para todos los proyectos futuros de mi plataforma

Regla 1 – No romper el sistema

No eliminar, renombrar, moveré ni reestructurar nada que afecte el funcionamiento actual sin evidencia técnica y sin que antes lo apruebe.

Eso incluye:

- Estructura de carpetas.
- Nombres de archivos.
- Flujo de navegación.
- Rutas.
- JavaScript.
- CSS.
- PHP.

- APIs.
- Base de datos.
- Automatizaciones.
- Integraciones.
- Asistente virtual.
- Calendly.
- WhatsApp.
- Formularios.
- Dashboard.
- Cualquier funcionalidad existente.

Mi filosofía será la misma que usaría en una empresa

Entender, Documentar, Luego: **Auditar.**, Y solamente después:

Proponer. Nunca al revés.

Tampoco vas a "modernizar por modernizar"

Esto es algo que veo muy seguido.

Hay quien dice:"Eso hay que hacerlo en React."No.

"Eso hay que hacerlo con Next.js." No.

"Eso debería ser Tailwind." No.

Mientras el sistema cumpla los objetivos de la plataforma, **no existe ninguna obligación técnica de migrarlo.**

Una migración solo se justifica si aporta un beneficio claro y medible.

Lo que sí buscar

No cambiar el comportamiento.

buscar:

- Riesgos.
- Deuda técnica.
- Seguridad.
- Escalabilidad.
- Mantenibilidad.
- Rendimiento.
- Integración con el resto de la plataforma.

Y cada recomendación tendrá que responder:

1. ¿Qué problema resuelve?
2. ¿Qué riesgo elimina?
3. ¿Qué impacto tiene?
4. ¿Qué efecto secundario produce?
5. ¿Vale la pena hacerlo?

Si no puedes responder esas cinco preguntas, **no recomendaré el cambio.**

segunda etapa

integrar el ecosistema:

-  Calendly (ya funcional).
-  Automatización de WhatsApp.
-  Asistente virtual con agentes en n8n.
-  Flujos entre n8n, Node-RED y la plataforma.

Tenerlo pendiente pero si hay que hacerlo terminando todo

Quiero recibir RFCs

Por ejemplo:

- RFC-PORT-001 → Publicación segura en GitHub.
- RFC-PORT-002 → Auditoría del Frontend.
- RFC-PORT-003 → APIs y Backend.
- RFC-PORT-004 → Migración Lenovo → Dell.
- RFC-PORT-005 → Dockerización del Portfolio.
- RFC-PORT-006 → Integración con Homer.
- RFC-PORT-007 → Automatización con n8n.
- RFC-PORT-008 → Dashboard de demostración (Quesinor y demás proyectos).

Cada RFC tendrá:

- Objetivo.
- Evidencia.
- Riesgos.
- Decisión.
- Plan de implementación.
- Validación.
- Criterios de aceptación.

Así quedará alineado con mi Manual

Tener en cuenta que Es la **primera evidencia de la metodología de trabajo**

insisto revisar el proyecto desde cuatro perspectivas simultáneamente:

1. Arquitecto de Software: estructura, modularidad, mantenibilidad.
2. Arquitecto de Infraestructura: despliegue, Docker, migración, operación.
3. Consultor de Ciberseguridad: exposición de información, GitHub, endurecimiento, superficie de ataque.
4. Cliente/Reclutador: claridad del mensaje, facilidad para encontrar evidencias y ejecutar demostraciones.

Hay una última regla que quiero acordar contigo

Durante la auditoría, voy a clasificar cada observación en una de estas categorías:

-  Crítico: debe resolverse antes de GitHub o antes de producción.
-  Alto: recomendable antes de publicar el portfolio.
-  Medio: mejora técnica, pero no bloquea.
-  Bajo: optimización o mejora futura.
-  No cambiar: decisiones que considero correctas y que deben mantenerse.

Esta última categoría ( No cambiar) es importante porque no quiero que el informe sea solo una lista de problemas. También quiero dejar documentado qué decisiones actuales son sólidas para que no las modifiques en el futuro "por probar algo nuevo".

portfolio contiene

(admin/ , api/ , assets/ , css/ , includes/ , js/ , logs/ , vendor/ , .htaccess , composer.json , composer.lock , composer.phar , index.html , README.md)

El proyecto ya está modularizado

Existe una separación entre:

- Frontend.
- Backend.
- Recursos.
- Librerías.
- Configuración del servidor.
- Dependencias PHP.

No **proponer una reestructuración completa**, porque no hace falta

Tiene README el proyecto puede empezar a documentarse correctamente desde el principio, alineado con el manual

Existe un directorio logs/ no deban terminar en GitHub

Existe vendor/ , composer.phar, revisarlos

Durante el análisis hay referencias compatibles con claves/API. Antes de GitHub revisar

línea por línea ese modulo

la plataforma ya empezó a integrar servicios externos
entonces propongo política para manejar:

- Tokens
- SID
- API Keys
- Secret Keys
- SMTP
- WhatsApp
- OpenAI
- Calendly

Todo esto deberá salir del código

Regla de Oro

Ningún proyecto podrá publicarse en GitHub sin pasar antes la auditoría "GitHub Ready".

No solo el portfolio.

También:

- Quesinor.
- Compras Públicas.
- Milagro de Vida.
- CRM.
- Automatizaciones.
- Laboratorios futuros.

Todos.

Credenciales SMTP expuestas `api/config.php` revisarlo

Ya que Todo está concentrado en `api/config.php` la migración hacia `.env` será limpia. Por ello esta así pero hay que revisarlo si no expone nada

Base de datos revisar Lo que cambia es dónde viven esos valores.

twilio/sdk esta en Compose revisar

PHPMailer el envío de correos ya está integrado.

revisar después es:

- TLS
- manejo de errores
- reintentos
- sanitización
- validación de correo
- rate limiting

Pero eso será otro RFC

`js/chatbot.js` revisarlo si había una API Key escrita directamente

Regla de Plataforma

Nunca existirá una contraseña dentro del código fuente.

No importa si es:

- Gmail
- Twilio
- WhatsApp
- OpenAI
- Calendly
- PostgreSQL
- MySQL
- MQTT

- Grafana
- Prometheus

Todas pasarán al mismo mecanismo.

Así, cuando llegue el proyecto de un cliente, simplemente reutilizaremos el estándar

Auditoría del BackOffice

admin/login.php revisalo
esas credenciales quedan:

- visibles para cualquier desarrollador con acceso al código,
- versionadas en Git,
- imposibles de rotar sin modificar el código,
- compartidas entre todos los entornos.

Esta así por estar en desarrollo

revisar :

- expiración,
- regeneración de ID,
- cookies seguras,
- SameSite,
- HttpOnly.

Ya que existe // CAMBIAR EN PRODUCCIÓN sabíamos que era temporal. Simplemente quedó pendiente.

El Dashboard Administración del Portfolio es realmente un Centro de Operaciones del Consultor

allí pueden convivir:

- visitas
- comentarios

- leads
- reuniones
- automatizaciones
- estado del laboratorio
- proyectos
- clientes

index.html

desde cuatro perspectivas:

- Reclutador.
- Cliente.
- CEO.
- Arquitecto de Infraestructura.

aquí ya no estoy revisando PH si no **la experiencia que tendrá un cliente cuando entre a contratarme.**

¿Este portfolio convence a un cliente para contratar una consultoría remota?

La estructura está orientada al cliente

El flujo del index.html es:

1. Hero
2. Métricas
3. Servicios
4. Casos de estudio
5. Reacciones/Testimonios
6. Tech Stack
7. Certificaciones
8. Why Remote

9. Contacto

No construí una página para hablar de mí. Construí un recorrido.

El usuario primero entiende:

Quién soy, Qué hago, Qué he hecho, Con qué tecnología, Qué certificaciones tengo,

Cómo contratarme, Ese orden creo que tiene sentido.

el menú habla de: Case Studies. Son **laboratorios demostrables**. Puede que en el futuro esa sección evolucione hacia algo que transmita mejor la capacidad de ejecutar proyectos en vivo.

La sección **Tech Stack** fue creada así es estratégica ya que cada tecnología está respaldada por un laboratorio. Eso significa que el Tech Stack **no es un listado de logotipos. Es un índice de evidencias.**

La sección **Credentials** las certificaciones tienen códigos QR verificables.

La sección **Why Remote** ya no es una página informativa.

La potencia hacia una pieza comercial. Su función no es explicar qué es el trabajo remoto.

Su función es responder la objeción:

"¿Por qué debería contratar a alguien que trabaja a distancia?" Es una decisión estratégica.

Reclutadores.

Clientes de consultoría.

ambos encuentran exactamente lo que buscan. el mismo recorrido consigue convencer a ambos, perfectamente sin duplicar la plataforma.

el **Hero**.

En los primeros 20–30 segundos, un CEO, un CTO o un reclutador entienden exactamente qué problema resuelves y por qué deberían seguir explorando tu portfolio? Esa primera pantalla es la que más impacto tiene sobre la tasa de conversión.

La frase principal dice, en esencia:

"Diseño e implementación de soluciones completas de TI: redes seguras, sistemas full-stack, infraestructura cloud y automatización industrial."

Intento Habla de resultados, no de tecnologías. Muchísima gente empieza diciendo:

PHP

Docker

AWS

Cisco

Python

Yo no. Empiezo hablando de lo que hago

qué puede contratarte un cliente Hay una diferencia enorme entre:

Sé hacer esto. Y Puedo resolver este problema de tu empresa.

Yo preferiría que un CEO entendiera inmediatamente: Ayudo a empresas a diseñar, implementar y operar infraestructura segura de forma completamente remota. Cambia el enfoque. Eso es lo que quiero orientarlo un poco más hacia **consultoría**. Habla de:

- IT Engineer
- sistemas
- infraestructura
- automatización
- consultoría remota

Eso está bastante bien para SEO. Creo

El nombre

Hoy aparece:

Edison Nicolas Guamialama Haro esto es susceptible

Cuando una empresa contrata una consultoría...

¿Qué quiere recordar?

El nombre de la persona o la marca. La marca también en un futuro es ENGH ya tengo el logo por el momento el protagonista del Hero debe ser:

la persona si me recomiendas la marca de consultoría. Debe ser sustentada

Aquí encontré otra oportunidad. Dice que hago muchas cosas.

Y es verdad. Pero precisamente por eso... el cerebro del visitante tarda un poco en clasificarme

Entonces

cuando revise mi posicionamiento profesional. Recuerdo que una de mis preocupaciones era: "Hago demasiadas cosas y no quiero parecer alguien que sabe un poco de todo."

Esa preocupación sigue vigente.

Mi objetivo será que el visitante piense:

"Este profesional tiene un alcance muy amplio, pero sé exactamente qué puede hacer por mi empresa."

Allí entra Real results from delivered projects... Eso conecta muy bien con mi filosofía.

Porque yo no vendo cursos.

No vendo certificaciones.

No vendo teoría.

Vendo evidencia. Y eso me parece uno de mis mayores diferenciales del portfolio.

Ya que **Cada afirmación importante debe poder demostrarse.**

Si dices: Experto en MPLS.

existe un laboratorio.

Si dices: Automatización industrial.

existe un caso de estudio.

Si dices: Cloud.

existe una evidencia. Esto convierte el portfolio en algo muy distinto de un CV por lo que evidencio en el código y en los documentos, **ya estoy construyendo exactamente esa filosofía.**

Hero no es de diseño.

Es de estrategia.

Cuando el reclutador o el Clientes de consultoría entro por primera vez...

Comprende que soy un ingeniero que puede ayudarte en 6 frentes demostrables para que te acompañe durante meses como consultor remoto.

Y esa, para mí, es la oportunidad de mejora más importante de la primera pantalla.

Numbers That Matter

los números no son bonitos, sino para responder una pregunta de negocio:

¿Estos números generan confianza suficiente para que un cliente continúe leyendo?

¿Esas métricas generan confianza o parecen marketing?

es uno de los elementos más persuasivos de todo el portfolio... o en uno de los más débiles, dependiendo de cómo esté sustentada.

se estructuro

La sección visualmente, no está sobrecargada, no distrae, Los números llaman la atención.

una métrica salta inmediatamente una pregunta.

¿De dónde sale? Estos datos ¿Dónde están?

Y aquí es donde mi portfolio tiene una ventaja enorme que existen.

Porque **yo sí puedes demostrarlo.**

Mis números **no son marketing.** Son navegables

Y creo que ese será uno de los mayores diferenciales de mi marca.

Considerando De negocio.

Los números son muy buenos. todavía parecen... estadísticas.

A esto se decidio convertirlos en: **indicadores verificables.**

Hay una diferencia enorme que pocos lo toman en cuenta **Evidencia.**

Entonces la credibilidad vale más que impresionar

Los números no están allí para vender ego.

Están allí para responder preguntas.

Por ejemplo.

Un cliente piensa:

¿Tiene experiencia? Tengo Proyectos

¿Tiene conocimientos? Tengo Tech Stack

¿Tiene respaldo? Dispongo de Certificaciones

¿Tiene resultados? Puedes descargar Casos de estudio

Cada sección responde una objeción distinta del cliente:

- Hero → ¿Quién eres y qué haces?
- Métricas → ¿Tienes experiencia demostrable?
- Servicios → ¿Qué puedes hacer por mí?
- Casos de estudio → ¿Dónde está la evidencia?
- Tech Stack → ¿Con qué tecnologías trabajas?
- Certificaciones → ¿Tienes respaldo profesional?
- Why Remote → ¿Por qué confiar en un servicio remoto?
- Contacto → ¿Cómo empezamos?

Eso me parece una estructura muy sólida.

Pensando en el cliente

Habla de psicología, consultoría y confianza.

Voy a ponerme en el lugar de un CTO o un CEO.

Soy el gerente de una empresa.

Necesito alguien que:

- diseñe infraestructura
- automatice procesos
- implemente redes
- documente

- opere remotamente

Busco en Google.

Encuentra mi portfolio.

Tengo aproximadamente **20 segundos**.

Aquí el Hero hace bien su trabajo.

Veo números.

Servicios. Empiezo a identificar su problema Ese cambio psicológico es enorme.

Casos de estudio. Veo proyectos reales.

Después de leer Qesinor... → Muéstreme eso.

Aquí es donde se abre las oportunidades por eelo esta asi estructurado

Google(plataformas), Portfolio, Qesinor, Caso de estudio, Video, PDF, Wiki ,

Aquí aparece algo nuevo, Solicitar demostración, Agenda, Zoom, Laboratorio, Dashboard, Proyecto funcionando, Cliente

es la diferencia El portfolio ya no es un sitio web, es un embudo de consultoría.

Y el laboratorio es la demostración. Aquí es donde entre el setup físico server, y sus contenedores , dashboard su funcionamiento

El Portfolio nunca debe mostrar todo.

Debe mostrar solamente lo suficiente para que el cliente quiera ver la demostración.

La demostración ocurre en la plataforma, no en la web, eso además protege propiedad intelectual

Cada caso de estudio debería tener dos niveles.

Nivel Público

Lo que cualquiera puede ver.

- descripción
- imágenes
- video corto

- tecnologías
- beneficios

Nivel Privado

Solamente durante la reunión. (Allí aparece: Wiki, Docker, Grafana, Logs, Código, Base de datos, Dashboards, Automatizaciones, Node-RED, n8n, Portainer, SSH, etc.)

Homer no es un dashboard bonito

Durante una reunión, no puedo hacer esto digitar: grafana.lab, portainer.lab, wiki.lab, crm.lab, prometheus.lab, etc. 21 contenedores levantar que jartera Pierdes cinco minutos pierdo concentración, el cliente espera., no es profesional.
homer no es para mi es para el cliente

Todo con un clic, y la reunión fluye.

En ningún momento de mi explicación dije: Quiero vender un portfolio.

Siempre dije: Quiero vender consultoría remota.

Entonces el portfolio tiene un objetivo muy específico: **Conseguir que el visitante dé el siguiente paso.**

empresa que todavía no existe legalmente, pero cuya plataforma ya se está construyendo.

"Confíame un problema tecnológico y yo puedo diseñarlo, implementarlo, operarlo, documentarlo y mantenerlo completamente de forma remota."

Tengo seis áreas cada área es una **línea de negocio**, no una categoría

Por ejemplo

Si un cliente entra buscando redes.

No debería sentir que está entrando a un portfolio , debería sentir que está entrando a una consultoría especializada en redes.

Si entra por automatización: Exactamente igual.

Si entra por IA: Lo mismo.

El portfolio debe adaptarse al motivo por el que llegó el cliente, no al revés.

Entonces apareció un concepto nuevo.

Y este creo que será el corazón de toda la plataforma.

"Evidence Driven Consulting"

No sé todavía si ese será el nombre definitivo pero sí la filosofía.

Todo lo que se dice...tiene evidencia.

Digo:

Automatización Industrial, Existe Quesinor.

Digo: Infraestructura Empresarial, Existe Dell(Grafana, Prometheus, Backups, Alta disponibilidad)

Digo: Ciberseguridad.(Existe el laboratorio, ISO, Auditorías, Pentesting)

Digo: Desarrollo.(Existe Compras Públicas., Microservicios, CRUD, Base de datos)

No hay teoría, hay evidencia, eso, sinceramente es un diferencial

Cuando un cliente termine de recorrer el portfolio

"Qué interesante." "Quiero hablar con él." Existe Calendly whatsapp,

...

